vLLM专题（十三）-结构化输出（Structured Outputs）

vLLM 支持使用 `outlines`、`lm-format-enforcer` 或 `xgrammar` 作为引导解码的后端来生成结构化输出。本文档展示了一些可用于生成结构化输出的不同选项示例。

一、在线服务（OpenAI API）

你可以使用 OpenAI 的 Completions 和 Chat API 生成结构化输出。

支持以下参数，这些参数必须作为额外参数添加：

- `guided_choice`：输出将完全匹配其中一个选项。
- `guided_regex`：输出将遵循正则表达式模式。
- `guided_json`：输出将遵循 JSON 模式。
- `guided_grammar`：输出将遵循上下文无关语法。
- `guided_whitespace_pattern`：用于覆盖默认的空白字符模式（适用于引导 JSON 解码）。
- `guided_decoding_backend`：用于选择引导解码后端。可以在后端名称后使用冒号分隔的列表提供特定于后端的选项。例如，`"xgrammar:no-fallback"` 将不允许 vLLM 在出错时回退到其他后端。

你可以在 OpenAI 兼容服务器页面 上查看支持的完整参数列表。

** 示例**
以下是每种情况的示例，从最简单的 `guided_choice` 开始：

1. 使用 `guided_choice`

```
from openai import OpenAI
client = OpenAI(
    base_url="http://localhost:8000/v1",
    api_key="-",
)

completion = client.chat.completions.create(
    model="Qwen/Qwen2.5-3B-Instruct",
    messages=[
        {
            "role": "user", "content": "Classify this sentiment: vLLM is wonderful!"}
    ],
    extra_body={
            "guided_choice": ["positive", "negative"]},
)
print(completion.choices[0].message.content)
```

2. 使用 `guided_regex 以下示例展示了如何使用` guided_regex` 生成符合正则表达式模板的电子邮件地址：

```
completion = client.chat.completions.create(
    model="Qwen/Qwen2.5-3B-Instruct",
    messages=[
        {

            "role": "user",
            "content": "Generate an example email address for Alan Turing, who works in E
nigma. End in .com and new line. Example result: alan.turing@enigma.com\n",
        }
    ],
    extra_body={
            "guided_regex": "\w+@\w+\.com\n", "stop": ["\n"]},
)
print(completion.choices[0].message.content)
```

3. 使用 `guided_json`
   在结构化文本生成中，最相关的功能之一是生成具有预定义字段和格式的有效 JSON。为此，我们可以通过两种方式使用 `guided_json` 参数：

- 直接使用 JSON Schema。
- 定义 Pydantic 模型，然后从中提取 JSON Schema（通常更简单）。

以下示例展示了如何使用 Pydantic 模型和 `guided_json` 参数：

```python
from pydantic import BaseModel
from enum import Enum

class CarType(str, Enum):
    sedan = "sedan"
    suv = "SUV"
    truck = "Truck"
    coupe = "Coupe"


class CarDescription(BaseModel):
    brand: str
    model: str
    car_type: CarType


json_schema = CarDescription.model_json_schema()

completion = client.chat.completions.create(
    model="Qwen/Qwen2.5-3B-Instruct",
    messages=[
        {

            "role": "user",
            "content": "Generate a JSON with the brand, model and car_type of the most ic
onic car from the 90's",
        }
    ],
    extra_body={
            "guided_json": json_schema},
)
print(completion.choices[0].message.content)
```

> **小贴士**
> 虽然并非严格必要，但通常在提示中指明需要生成 JSON 以及模型应如何填充字段会更好。在大多数情况下，这可以显著改善结果。

最后是 `guided_grammar`，这可能是最难使用但功能最强大的选项，因为它允许我们定义完整的语言，例如 SQL 查询。它通过使用上下文无关的 EBNF 语法来实现，例如我们可以用它来定义一种特定格式的简化 SQL 查询，如下例所示：

```
simplified_sql_grammar = """
    ?start: select_statement

    ?select_statement: "SELECT " column_list " FROM " table_name

    ?column_list: column_name ("," column_name)*

    ?table_name: identifier

    ?column_name: identifier

    ?identifier: /[a-zA-Z_][a-zA-Z0-9_]*/
"""

completion = client.chat.completions.create(
    model="Qwen/Qwen2.5-3B-Instruct",
    messages=[
        {

            "role": "user",
            "content": "Generate an SQL query to show the 'username' and 'email' from the
 'users' table.",
        }
    ],
    extra_body={
            "guided_grammar": simplified_sql_grammar},
)
print(completion.choices[0].message.content)
```

完整示例：`examples/online_serving/openai_chat_completion_structured_outputs.py`

二、实验性自动解析（OpenAI API）

本节介绍 OpenAI 对 `client.chat.completions.create()` 方法的 beta 封装，该封装提供了与 Python 特定类型更丰富的集成。

截至撰写时（`openai==1.54.4`），这是 OpenAI 客户端库中的“beta”功能。代码参考可以在此处找到。

以下示例中，vLLM 使用 `vllm serve meta-llama/Llama-3.1-8B-Instruct` 启动。

1. 使用 Pydantic 模型生成结构化输出的简单示例

```python
from pydantic import BaseModel
from openai import OpenAI


class Info(BaseModel):
    name: str
    age: int


client = OpenAI(base_url="http://0.0.0.0:8000/v1", api_key="dummy")
completion = client.beta.chat.completions.parse(
    model="meta-llama/Llama-3.1-8B-Instruct",
    messages=[
        {
            "role": "system", "content": "You are a helpful assistant."},
        {
            "role": "user", "content": "My name is Cameron, I'm 28. What's my name and ag
e?"},
    ],
    response_format=Info,
    extra_body=dict(guided_decoding_backend="outlines"),
)

message = completion.choices[0].message
print(message)
assert message.parsed
print("Name:", message.parsed.name)
print("Age:", message.parsed.age)
```

输出：

```
ParsedChatCompletionMessage[Testing](content='{"name": "Cameron", "age": 28}', refusal=No
ne, role='assistant', audio=None, function_call=None, tool_calls=[], parsed=Testing(name=
'Cameron', age=28))
Name: Cameron
Age: 28
```

2. 使用嵌套 Pydantic 模型处理分步数学解决方案的复杂示例

```python
from typing import List
from pydantic import BaseModel
from openai import OpenAI


class Step(BaseModel):
    explanation: str
    output: str


class MathResponse(BaseModel):
    steps: List[Step]
    final_answer: str


client = OpenAI(base_url="http://0.0.0.0:8000/v1", api_key="dummy")
completion = client.beta.chat.completions.parse(
    model="meta-llama/Llama-3.1-8B-Instruct",
    messages=[
        {
            "role": "system", "content": "You are a helpful expert math tutor."},
        {
            "role": "user", "content": "Solve 8x + 31 = 2."},
    ],
    response_format=MathResponse,
    extra_body=dict(guided_decoding_backend="outlines"),
)

message = completion.choices[0].message
print(message)
assert message.parsed
for i, step in enumerate(message.parsed.steps):
    print(f"Step #{
              i}:", step)
print("Answer:", message.parsed.final_answer)
```

输出：

```
ParsedChatCompletionMessage[MathResponse](content='{ "steps": [{ "explanation": "First, l
et\'s isolate the term with the variable \'x\'. To do this, we\'ll subtract 31 from both
sides of the equation.", "output": "8x + 31 - 31 = 2 - 31"}, { "explanation": "By subtrac
ting 31 from both sides, we simplify the equation to 8x = -29.", "output": "8x = -29"}, {
 "explanation": "Next, let\'s isolate \'x\' by dividing both sides of the equation by 8."
, "output": "8x / 8 = -29 / 8"}], "final_answer": "x = -29/8" }', refusal=None, role='ass
istant', audio=None, function_call=None, tool_calls=[], parsed=MathResponse(steps=[Step(e
xplanation="First, let's isolate the term with the variable 'x'. To do this, we'll subtra
ct 31 from both sides of the equation.", output='8x + 31 - 31 = 2 - 31'), Step(explanatio
n='By subtracting 31 from both sides, we simplify the equation to 8x = -29.', output='8x
= -29'), Step(explanation="Next, let's isolate 'x' by dividing both sides of the equation
 by 8.", output='8x / 8 = -29 / 8')], final_answer='x = -29/8'))
Step #0: explanation="First, let's isolate the term with the variable 'x'. To do this, we
'll subtract 31 from both sides of the equation." output='8x + 31 - 31 = 2 - 31'
Step #1: explanation='By subtracting 31 from both sides, we simplify the equation to 8x =
 -29.' output='8x = -29'
Step #2: explanation="Next, let's isolate 'x' by dividing both sides of the equation by 8
." output='8x / 8 = -29 / 8'
Answer: x = -29/8
```

三、离线推理

离线推理允许使用相同类型的引导解码。要使用它，我们需要通过 `SamplingParams` 类中的 `GuidedDecodingParams` 来配置引导解码。`GuidedDecodingParams` 中的主要可用选项包括：

- json
- regex
- choice
- grammar
- backend
- whitespace_pattern

这些参数可以像上面在线服务示例中的参数一样使用。以下是使用 `choices` 参数的示例：

```
from vllm import LLM, SamplingParams
from vllm.sampling_params import GuidedDecodingParams

llm = LLM(model="HuggingFaceTB/SmolLM2-1.7B-Instruct")

guided_decoding_params = GuidedDecodingParams(choice=["Positive", "Negative"])
sampling_params = SamplingParams(guided_decoding=guided_decoding_params)
outputs = llm.generate(
    prompts="Classify this sentiment: vLLM is wonderful!",
    sampling_params=sampling_params,
)
print(outputs[0].outputs[0].text)
```

完整示例如下：`structured_outputs.py`

```
# SPDX-License-Identifier: Apache-2.0

from enum import Enum

from pydantic import BaseModel

from vllm import LLM, SamplingParams
from vllm.sampling_params import GuidedDecodingParams

llm = LLM(model="Qwen/Qwen2.5-3B-Instruct", max_model_len=100)

# Guided decoding by Choice (list of possible options)
guided_decoding_params = GuidedDecodingParams(choice=["Positive", "Negative"])
sampling_params = SamplingParams(guided_decoding=guided_decoding_params)
outputs = llm.generate(
    prompts="Classify this sentiment: vLLM is wonderful!",
    sampling_params=sampling_params,
)
print(outputs[0].outputs[0].text)

# Guided decoding by Regex
guided_decoding_params = GuidedDecodingParams(regex="\w+@\w+\.com\n")
sampling_params = SamplingParams(guided_decoding=guided_decoding_params,
                                 stop=["\n"])
prompt = ("Generate an email address for Alan Turing, who works in Enigma."
          "End in .com and new line. Example result:"
          "alan.turing@enigma.com\n")
outputs = llm.generate(prompts=prompt, sampling_params=sampling_params)
print(outputs[0].outputs[0].text)


# Guided decoding by JSON using Pydantic schema
```

```
class CarType(str, Enum):
    sedan = "sedan"
    suv = "SUV"
    truck = "Truck"
    coupe = "Coupe"


class CarDescription(BaseModel):
    brand: str
    model: str
    car_type: CarType


json_schema = CarDescription.model_json_schema()

guided_decoding_params = GuidedDecodingParams(json=json_schema)
sampling_params = SamplingParams(guided_decoding=guided_decoding_params)
prompt = ("Generate a JSON with the brand, model and car_type of"
          "the most iconic car from the 90's")
outputs = llm.generate(
    prompts=prompt,
    sampling_params=sampling_params,
)
print(outputs[0].outputs[0].text)

# Guided decoding by Grammar
simplified_sql_grammar = """
    ?start: select_statement

    ?select_statement: "SELECT " column_list " FROM " table_name

    ?column_list: column_name ("," column_name)*

    ?table_name: identifier

    ?column_name: identifier

    ?identifier: /[a-zA-Z_][a-zA-Z0-9_]*/
"""
guided_decoding_params = GuidedDecodingParams(grammar=simplified_sql_grammar)
sampling_params = SamplingParams(guided_decoding=guided_decoding_params)
prompt = ("Generate an SQL query to show the 'username' and 'email'"
          "from the 'users' table.")
outputs = llm.generate(
    prompts=prompt,
    sampling_params=sampling_params,
)
print(outputs[0].outputs[0].text)
```